

操作系统课程设计 实验报告

姓名：王宁波 班级：1 班 学号：202100130176

2024-5-21

目录

Nachos 系统的安装与调试（实验 1）	3
实验目的.....	3
实验任务.....	3
实验步骤与内容.....	3
结论分析与体会.....	5
Nachos 的 Makefiles（实验 2）	8
实验目的.....	8
实验要求.....	8
实验步骤与内容.....	8
结论分析与体会.....	9
利用信号量实现线程同步（实验 3）	9
实验目的.....	10
实验要求.....	10
实验步骤与内容.....	10
结论分析与体会.....	13
Nachos 的文件系统（实验 4）	14
实验目的.....	14
实验步骤与内容.....	15
结论分析与体会.....	22
扩展 Nachos 的文件系统（实验 5）	23
实验目的.....	23
实验要求.....	23
实验步骤与内容.....	24
结论分析与体会.....	28
Nachos 用户程序与系统调用（实验 6）	29
实验目的.....	29
实验要求.....	30
实验步骤与内容.....	30
结论分析与体会.....	33
地址空间的扩展（实验 7）	33

实验目的.....	34
实验要求.....	34
实验步骤与内容.....	34
结论分析与体会.....	37
系统调用 Exec()与 Exit() (实验 8)	37
实验目的.....	37
实验要求.....	38
实验步骤与内容.....	38
1. 实验准备	38
2. 系统调用的实现.....	39
3. 系统调用的测试.....	49
结论分析与体会.....	51

Nachos 系统的安装与调试（实验 1）

实验题目： Nachos 系统的安装与调试		学号： 202100130176
日期： 2024. 3. 15	班级： 计科一班	姓名： 王宁波
Email： 2197686837@qq. com		
实验目的： 1. 正确安装、编译、测试 Nachos 系统，理解 Nachos 系统的组织结构； 2. 安装、测试 gcc MIPS 交叉编译器； 3. 熟悉 Nachos 启动时一些常用的命令行参数 4. 掌握利用 Linux 调试工具 GDB 调试跟踪 Nachos 的执行过程； 5. 阅读 Nachos 的相关源代码 ■ 理解 Nachos 的启动、关机过程 ■ 了解 Nachos 的第一个线程的产生方法、后续线程的创建方法，以及 idler 进程的创建与使用； ■ 进而了解 Linux、Windows 等实际操作系统中第一个进程的产生过程，以及 idler 进程的创建与作用； ■ 结合 Nachos 的测试结果，理解 Nachos 的线程调度方法； ■ 结合定时器 Timer 及定时器中断，理解 RR 调度算法的实现方法； ■ 通过跟踪 Nachos 的 C++程序，以及上下文切换部分的汇编代码，理解 Nachos 中线程上下文切换的过程。（可以通过 GDB） ■ 理解启动时相关参数的含义		
实验任务： ● Install the LINUX operating system (optional) ✓ 如果安装的是32位Linux，跳过“在 Ubuntu 64 位上编译 Nachos”这一步 ✓如果安装的是64位Linux，必需完成“在 Ubuntu 64 位上编译 Nachos”这一过程 ● Install the Nachos system ● Install gcc mips cross compiler ● Compile and test the Nachos system installed ● Exercise with gdb ● Context switch in Nachos		
实验步骤与内容： 1. 安装 Nachos 及交叉编译器 测试交叉编译器		

```
>>> Converting to noff file: arch/unknown-i386-linux/bin/sort.noff <<<
../bin/arch/unknown-i386-linux/bin/coff2noff arch/unknown-i386-linux/objects/sort.coff arch/unknown-i386-linux/objects/sort.noff
numsections 3
Loading 3 sections:
".text", filepos 0xd0, mempos 0x0, size 0x2c0
".data", filepos 0x390, mempos 0x2c0, size 0x0
".bss", filepos 0x0, mempos 0x2c0, size 0x1000
ln -sf arch/unknown-i386-linux/bin/sort.noff sort.noff
```

2. 测试 Nachos 的基本内核

在目录 `./threads` 依次运行: `make clean, make`

```
>>> Linking arch/unknown-i386-linux/bin/nachos <<<
g++ -m32 arch/unknown-i386-linux/objects/main.o arch/unknown-i386-linux/objects/list.o arch/unknown-i386-linux/objects/schedule
r.o arch/unknown-i386-linux/objects/synch.o arch/unknown-i386-linux/objects/synchlist.o arch/unknown-i386-linux/objects/system.
o arch/unknown-i386-linux/objects/thread.o arch/unknown-i386-linux/objects/utility.o arch/unknown-i386-linux/objects/threadtest
.o arch/unknown-i386-linux/objects/synchtest.o arch/unknown-i386-linux/objects/interrupt.o arch/unknown-i386-linux/objects/sysd
ep.o arch/unknown-i386-linux/objects/stats.o arch/unknown-i386-linux/objects/timer.o arch/unknown-i386-linux/objects/switch-lin
ux.o -o arch/unknown-i386-linux/bin/nachos
/usr/bin/ld: arch/unknown-i386-linux/objects/switch-linux.o: warning: relocation against `_eax_save' in read-only section `.tex
t'
/usr/bin/ld: warning: creating DT_TEXTREL in a PIE
ln -sf arch/unknown-i386-linux/bin/nachos nachos
```

运行 `./nachos`

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/threads$ ./nachos
*** thread 0 looped 0 times
*** thread 1 looped 0 times
*** thread 0 looped 1 times
*** thread 1 looped 1 times
*** thread 0 looped 2 times
*** thread 1 looped 2 times
*** thread 0 looped 3 times
*** thread 1 looped 3 times
*** thread 0 looped 4 times
*** thread 1 looped 4 times
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 130, idle 0, system 130, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

3. 测试 Nachos 的其它模块

`./filesystems`

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ ./nachos
*** thread 0 looped 0 times
*** thread 1 looped 0 times
*** thread 0 looped 1 times
*** thread 1 looped 1 times
*** thread 0 looped 2 times
*** thread 1 looped 2 times
*** thread 0 looped 3 times
*** thread 1 looped 3 times
*** thread 0 looped 4 times
*** thread 1 looped 4 times
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 1190, idle 1000, system 190, user 0
Disk I/O: reads 2, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

`./userprog`

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/userprog$ ./nachos
```

```
*** thread 0 looped 0 times
*** thread 1 looped 0 times
*** thread 0 looped 1 times
*** thread 1 looped 1 times
*** thread 0 looped 2 times
*** thread 1 looped 2 times
*** thread 0 looped 3 times
*** thread 1 looped 3 times
*** thread 0 looped 4 times
*** thread 1 looped 4 times
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!
```

```
Ticks: total 130, idle 0, system 130, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
```

Monitor

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/monitor$ ./nachos
```

```
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!
```

```
Ticks: total 590, idle 0, system 590, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
```

```
Cleaning up...
```

Vm

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/vm$ ./nachos
```

```
*** thread 0 looped 0 times
*** thread 1 looped 0 times
*** thread 0 looped 1 times
*** thread 1 looped 1 times
*** thread 0 looped 2 times
*** thread 1 looped 2 times
*** thread 0 looped 3 times
*** thread 1 looped 3 times
*** thread 0 looped 4 times
*** thread 1 looped 4 times
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!
```

```
Ticks: total 130, idle 0, system 130, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
```

```
Cleaning up...
```

4. 熟悉 gdb

结论分析与体会:

1. 理解 Nachos 的启动与关闭流程

从 `./threads/main.cc` 开始, 分析 Nachos 源代码

理解 Nachos 的启动与关闭流程

- (1) 初始化 Nachos 的设备与内核
调用 `Initialize(int argc, char **argv)`
- (2) 对 Nachos 内核进行测试(线程的创建及并发执行)
调用 `ThreadTest()` 测试内核是否工作正常, 测试线程创建及多线程并发交替执行过程
- (3) 测试 Nachos 的锁机制及条件变量
- (4) 处理 Nachos 其它的一些命令行参数

(5) 终止主线程, Nachos 退出

main() 函数退出之前, 执行 currentThread->Finish() 终止主线程, 并调度执行就绪进程; 当所有就绪线程执行结束, 且无中断要处理, 才退出 Nachos。

理解一些常用的 Nachos 启动参数 (参见 ../code/threads/thread.cc)

2. 根据运行结果, 理解

Nachos 中第一个线程是如何产生的?

主函数调用 Initialize(argc, argv) 后, 创建主线程 main, 并作为当前运行的线程

```
currentThread = new Thread("main");
currentThread->setStatus(RUNNING);
```

Nachos 中除第一个线程之外的其它线程是如何创建的?

主函数调用 ThreadTest 后, 主线程 "main" 创建了一个子线程 "forked thread"

```
ThreadTest()
{
    DEBUG('t', "Entering SimpleTest");

    Thread *t = new Thread("forked thread");

    t->Fork(SimpleThread, 1);
    SimpleThread(0);
}
```

线程调度的机理与实现

main() 函数退出之前, 执行 currentThread->Finish() 终止主线程, Thread::Finish() 调用了 Thread::Sleep(), 引起线程调度, 如果此时就绪队列中尚有就绪进程, 则调度执行之; 当该就绪线程执行结束后, 也会自动执行 Thread::Finish(), 致使所有的就绪线程都会被依次调度执行; 如果主线程执行 Thread::Finish() 时就绪队列为空, 或者最后一个就绪线程执行结束后终止, 都会调用 Thread::Sleep()

3. 利用 gdb 工具跟踪 Nachos 的执行过程以及跟踪上下文切换函数 SWITCH() 及函数 ThreadRoot() 的执行过程。

熟悉跟踪过程后, 回答下列问题:

(1) 在你所生成的 Nachos 系统中, 下述函数的地址是多少? 并说明找到这些函数地址的过程及方法。

- i. InterruptEnable()
- ii. SimpleThread()
- iii. ThreadFinish()
- iv. ThreadRoot()

```
(gdb) b InterruptEnable
Breakpoint 1 at 0x2f2c: file thread.cc, line 242.
(gdb) b SimpleThread
Breakpoint 2 at 0x315b: file threadtest.cc, line 29.
(gdb) b ThreadFinish
Breakpoint 3 at 0x2f04: file thread.cc, line 241.
(gdb) b ThreadRoot
Breakpoint 4 at 0x4c58
```

(gdb) b function, 在函数 function 的入口设置断点, 并输出断点号及函数地址
可以设置函数 InterruptEnable()、SimpleThread()、ThreadFinish() 及 ThreadRoot() 的断点, 输出的断点地址就是函数地址

(2) 下述线程对象的地址是多少? 并说明找到这些对象地址的过程及方法。

i. the main thread of the Nachos

```
(gdb) b 150
Breakpoint 2 at 0x56557874: file system.cc, line 150.
(gdb) c
Continuing.

Breakpoint 2, Initialize (argc=0, argv=0xffffd3c8) at system.cc:150
150     currentThread->setStatus(RUNNING);
(gdb) p currentThread
$1 = (Thread *) 0x56563ca0
```

0x56563ca0

ii. the forked thread created by the main thread

```
(gdb) b 49
Breakpoint 4 at 0x5655820d: file threadtest.cc, line 49.
(gdb) c
Continuing.

Breakpoint 4, ThreadTest () at threadtest.cc:49
49     SimpleThread(0);
(gdb) p t
$2 = (Thread *) 0x56563d00
```

0x56563d00

(3) 当主线程第一次运行 SWITCH() 函数, 执行到函数 SWITCH() 的最后一条指令 ret 时, CPU 返回的地址是多少? 该地址对应程序的什么位置?

```
0x56559caf in SWITCH ()
(gdb) si
0x56559cb2 in SWITCH ()
(gdb) info r
eax      0x56559c54      1448451156
ecx      0x56557f1b      1448443675
edx      0x1             1
ebx      0x0             0
esp      0x56568d50      0x56568d50
ebp      0x0             0x0
```

0x56559c54, 对应 ThreadRoot() 中的第一条指令

(4) 当调用 Fork() 新建的线程首次运行 SWITCH() 函数时, 当执行到函数 SWITCH() 的最后一条指令 ret 时, CPU 返回的地址是多少? 该地址对应程序的什么位置?


```
0x56559cb2 in SWITCH ()
(gdb) info r
eax      0x565569d2      1448438226
ecx      0xc             12
edx      0x9             9
ebx      0x5655df40      1448468288
esp      0xffffd1ec      0xffffd1ec
```

0x565569d2 即第二次 SWITCH 函数结束的返回地址 Scheduler::Run() 函数中

Nachos 的 Makefiles（实验 2）

实验题目： 实验二 Nachos 的 Makefiles		学号： 202100130176
日期： 2024. 3. 20	班级： 计科一班	姓名： 王宁波
Email： 2197686837@qq. com		
实验目的：熟悉 Makefile 的使用 理解 Nachos 中 Makefile 的使用		
实验要求： ● 在 lab2 中，利用 touch 命令更改已存在文件的时间，熟悉 make 文件的设置与使用 ● 将实验 3 需要修改的文件，及涉及到的文件复制到工作目录../lab3 中，以在../lab3 中完成该实验； ● 将实验 4、5 需要修改的文件，及涉及到的文件复制到工作目录../lab5 中，以在../lab5 中完成该实验； ● 将实验 6、7、8 需要修改的文件，及涉及到的文件复制到工作目录../lab7-8 中，以在../lab7-8 中完成该实验；		
实验步骤与内容： 1. Lab2 利用 touch 更新 threads/scheduler.h 的时间，在 code/lab2 下重新运行 make 命令，测试对 threads/scheduler.h 的改变是否影响 Nachos 的生成。 <pre>wnb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/threads\$ touch ../threads/scheduler.h wnb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/threads\$ touch ../threads/scheduler.h wnb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/threads\$</pre>		

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/lab2$ make
make: "arch/unknown-i386-linux/bin/nachos"已是最新。
```

利用 touch 命令修改 lab2 目录中的 scheduler.h 的时间为当前时间，make 程序会认为 scheduler.h 已被修改，运行 make 时会重新编译 lab2 中与 scheduler.h 关联的所有 C++源程序 lab2/scheduler.cc

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/lab2$ touch scheduler.h
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/lab2$ make
>>> Building dependency file for ../machine/timer.cc <<<
>>> Building dependency file for ../machine/sysdep.cc <<<
>>> Building dependency file for ../machine/interrupt.cc <<<
>>> Building dependency file for synctest.cc <<<
>>> Building dependency file for threadtest.cc <<<
>>> Building dependency file for thread.cc <<<
>>> Building dependency file for system.cc <<<
>>> Building dependency file for synch.cc <<<
>>> Building dependency file for scheduler.cc <<<
>>> Building dependency file for main.cc <<<
>>> Compiling main.cc <<<
g++ -m32 -g -Wall -Wshadow -I../lab2 -I../threads -I../machine -DTHREADS -DHOST_i386 -DHOST_LINUX -DCHANGED -c -o arch/unknown-i386-linux/objects/main.o main.cc
>>> Compiling scheduler.cc <<<
g++ -m32 -g -Wall -Wshadow -I../lab2 -I../threads -I../machine -DTHREADS -DHOST_i386 -DHOST_LINUX -DCHANGED -c -o arch/unknown-i386-linux/objects/scheduler.o scheduler.cc
```

结论分析与体会:

1. 对上述实验的解释

在 lab2 中运行 make 时，根据 lab2 中 Makefile.local 的设置（INCPATH += -I../lab2 -I../threads -I../machine），会按顺序依次从 lab2、threads 与 machine 目录中查找使用涉及到的头文件，并将直接或间接包含 threads/scheduler.h 的 c++源文件重新编译。

由于 scheduler.cc 已经在 lab2 中，因此 make 会编译 lab2/scheduler.cc，不会编译 threads/scheduler.cc。如果将 threads 目录下涉及到 threads/scheduler.h（也就涉及到 threads/system.h）所有头文件以及 c++源程序复制到 lab2 目录后，在 lab2 目录中，先用 touch ../threads/scheduler.h 更新 threads/scheduler.h，再运行 make，make 程序都会首先查找使用 lab2 中的相应文件（.h 与 .cc），就不会涉及到 code/threads 目录中的内容，也就不会重新编译它们。

利用信号量实现线程同步（实验 3）

实验题目： 实验三 利用信号量实现线程同步		学号： 202100130176
日期： 2024. 3. 29	班级： 计科一班	姓名： 王宁波
Email： 2197686837@qq. com		

实验目的:

- 理解生产者-消费者模型在 Nachos 中的实现方法与所需的数据结构;
- 掌握 Nachos 中如何创建线程;
- 理解 Nachos 中信号量是如何实现的;
- 掌握如何创建与使用 Nachos 的信号量
- 掌握 Nachos 中是如何利用信号量实现 producer/consumer problem;
- 理解 Nachos 中轮转法 (RR) 线程调度算法的实现方法
- 掌握 Nachos 中如何测试与调试程序

实验要求:

- 阅读 ../code/lab3 目录下的 ring.h、ring.cc、main.cc 及 prodcons++.cc
- 在理解它们工作机理的基础上, 补充 ../code/lab3 中提供的代码, 利用 Nachos 实现的信号量实现 producer/consumer problem。
✓ 主要是在 prodcons++.cc 中补充
- 分析 ../threads/thread.cc 中 Thread::Fork(), 理解内核创建线程的过程;
- 分析 ../threads/threadtest.cc, 掌握利用 Thread::Fork() 创建线程的方法;
- 分析 ../threads/synch.cc, 理解 Nachos 中信号量是如何实现的; 可进一步理解管程的实现方法;
- 分析 ../monitor/prodcons++.cc, 理解信号量的创建与使用方法; (为 lab3 提供实现思路)
- 分析 Thread::Fork(), Thread::Yield(), Thread::Sleep(), Thread::Finish(), Scheduler::Scheduler::ReadyToRun(), Scheduler::FindNextToRun(), Scheduler::Run()、switch-linux.s 等相关内容, 理解线程调度及上下文切换的工作过程;
- 分析 Nachos 对参数 -rs 的处理过程, 理解时钟中断的实现, 以及利用时钟中断实现 RR 调度算法;

实验步骤与内容:

1. 设计与实现

1.1 主控线程: ProdCons

```

void
ProdCons()
{
    int i;
    DEBUG('t', "Entering ProdCons");
    // 在这里放置构造所有信号量的代码。
    mutex = new Semaphore("mutex", 1);
    nfull = new Semaphore("nfull", 0);
    nempty = new Semaphore("nempty", BUFF_SIZE);
    // 在这里放置使用大小为BUFF_SIZE的环形缓冲区构造一个环形缓冲区对象的代码。
    ring = new Ring(BUFF_SIZE);
    // 创建并启动N_PROD个生产者线程
    for (i=0; i < N_PROD; i++)
    {
        // 这个语句用于形成一个字符串，用作生产者i的名称。
        sprintf(prod_names[i], "producer_%d", i);

        // 使用prod_names[i]作为名称，使用整数i作为"Producer"函数的参数，创建并启动一个新的生产者线程。
        producers[i] = new Thread(prod_names[i]);
        producers[i]->Fork(Producer, i);
    };
    // 创建并启动N_CONS个消费者线程
    for (i=0; i < N_CONS; i++)
    {
        // 这个语句用于形成一个字符串，用作消费者i的名称。
        sprintf(cons_names[i], "consumer_%d", i);
        // 使用cons_names[i]作为名称，使用整数i作为"Consumer"函数的参数，创建并启动一个新的消费者线程。
        consumers[i] = new Thread(cons_names[i]);
        consumers[i]->Fork(Consumer, i);
    };
}

```

完成了下列任务

- 声明、创建、初始化信号量：互斥信号量 mutex 和同步信号量 nfull 和 nempty
- 声明、创建、初始化缓冲池
- 创建生产者线程：顺序创建调用对应数量的生产者
- 创建消费者线程：顺序创建调用对应数量的生产者

1.2 生产者线程

```

void
Producer(_int which)
{
    printf("Producer %d running\n", which);
    int num;
    slot *message = new slot(0,0);

    // 这个循环用于生成N_MESSG条消息，并通过调用ring->Put(message)将其放入环形缓冲区。
    // 每条消息携带一个由整数"num"表示的消息ID，该消息ID应该放入槽的"value"字段中。
    // 它还应该携带生产者线程的ID，以便消费者线程稍后知道是哪个生产者生成了该消息。
    // 在调用ring->Put(message)之前和之后，需要进行同步。参考教材第182页的算法。
    for (num = 0; num < N_MESSG; num++) {
        // 在这里放置准备消息的代码。
        message->value = num;
        message->thread_id = which;

        // 在调用ring->Put(message)之前放置同步代码。
        nempty->P();
        mutex->P(); // 注意次序否则死锁

        ring->Put(message);

        // 在调用ring->Put(message)之后放置同步代码。
        mutex->V();
        nfull->V();
    }
    // if (sharing) currentThread->Yield();
}

```

完成了下列任务：

- ```

while (终止条件)
{
 初始化一个消息 msg;
 申请一个空缓冲区； nempty->P();
 申请缓冲池（对缓冲池互斥访问）； mutex->P();
 将消息 msg 放入缓存； put
 释放缓冲池； mutex->V()
 通知消费者； nfull->V();
}

```

## 1.3 消费者线程

```

void
Consumer(_int which)
{
 char str[MAXLEN];
 char fname[LINLEN];
 int fd;
 slot *message = new slot(0,0);
 // 为该消费者线程形成一个输出文件名。
 sprintf(fname, "tmp_%d", which); // 该消费者接收到的所有消息将记录在该文件中。
 if ((fd = creat(fname, 0600)) == -1) // 创建一个文件。注意，这是一个UNIX系统调用。
 {
 perror("creat: file create failed");
 exit(1);
 }
 // 死循环，不会执行完成，必须通过调度器调度执行其他消费者线程
 for (;;) {
 nfull->P();
 mutex->P();
 ring->Get(message);
 mutex->V();
 nempty->V();
 // 形成一个字符串来记录消息
 sprintf(str, "producer id --> %d; Message number --> %d;\n",
 message->thread_id,
 message->value);
 // stdout
 printf("consumer[%d] get producer id --> %d; Message number --> %d;\n", which,
 message->thread_id,
 message->value);
 // 将该字符串写入该消费者的输出文件中。
 // 注意，这是另一个UNIX系统调用。
 if (write(fd, str, strlen(str)) == -1) {

```

完成了下列任务：

创建一个该线程对应的文本文件，用于存放从缓冲池读出的消息；

while（终止条件）

{

    申请一个满缓冲区； nfull->P()；

    申请缓冲池（对缓冲池互斥访问）； mutex->P()；

    从缓存中取出消息 msg； ring->Get(message)；

    释放缓冲池； mutex->V()；

    通知消费者； nempty->V()；

    将取出的消息写到该线程对应的文本文件中；

}

    关闭文本文件；

## 2. 测试结果

```

wnb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/lab3$./nachos
Producer 0 running
Producer 1 running
consumer[0] get producer id --> 0; Message number --> 0
consumer[0] get producer id --> 0; Message number --> 1
consumer[0] get producer id --> 0; Message number --> 2
consumer[0] get producer id --> 0; Message number --> 3
consumer[0] get producer id --> 1; Message number --> 0
consumer[0] get producer id --> 1; Message number --> 1
consumer[0] get producer id --> 1; Message number --> 2
consumer[0] get producer id --> 1; Message number --> 3
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 730, idle 0, system 730, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...

```

此时只有 tmp\_0 中保存了上图内容，即会出现所有消息都在同一个文件中，而另一个文件中消息为空，原因是两个消费者线程顺序执行，而没有并发执行。此时需要采用抢先式调度，例如 RR 使消费者线程并发执行。

解决方法：启动 Nachos 是加上参数-rs，即 nachos -rs，可触发定时器中断

```
wmb@wmb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/lab3$./nachos -rs 6
Producer 0 running
Producer 1 running
consumer[0] get producer id --> 0; Message number --> 0
consumer[0] get producer id --> 0; Message number --> 1
consumer[0] get producer id --> 0; Message number --> 2
consumer[1] get producer id --> 1; Message number --> 0
consumer[0] get producer id --> 0; Message number --> 3
consumer[1] get producer id --> 1; Message number --> 1
consumer[1] get producer id --> 1; Message number --> 2
consumer[1] get producer id --> 1; Message number --> 3
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 817, idle 27, system 790, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

```
nachos-3.4-SDU > code > lab3 > tmp_0
1 producer id --> 0; Message number --> 0;
2 producer id --> 0; Message number --> 1;
3 producer id --> 0; Message number --> 2;
4 producer id --> 0; Message number --> 3;
5
```

```
nachos-3.4-SDU > code > lab3 > tmp_1
1 producer id --> 1; Message number --> 0;
2 producer id --> 1; Message number --> 1;
3 producer id --> 1; Message number --> 2;
4 producer id --> 1; Message number --> 3;
5
```

## 结论分析与体会：

1. Nachos 中轮转法（RR）线程调度算法的实现方法  
答：

```
else if (!strcmp(*argv, "-rs")) { // 随机数种子
 ASSERT(argc > 1); // 断言
 RandomInit(atoi(*(argv + 1))); // 初始化随机数种子

 randomYield = TRUE; // 随机产生时间片
 argCount = 2; // 本轮次到解析参数的个数
```

首先是函数 Initilize() 对参数 rs 进行处理， RandomInit() 初始化随机数种子，并将 randomYield 设为 true

```
// RR算法的硬件依赖
if (randomYield) // start the timer (if needed)
timer = new Timer(TimerInterruptHandler, 0, randomYield);
```

接着创建定时器设备 timer

```
static void
TimerInterruptHandler(_int dummy)
{
 if (interrupt->getStatus() != IdleMode) // 若不是空闲模式
 interrupt->YieldOnReturn(); // 产生时间片, 将yieldOnReturn标志设为true
}
```

定时器设备被设置为定期中断 CPU（每个 TimerTicks 一次），每次发生定时器中断时，都会调用此例程，产生时间片，将 yieldOnReturn 标志设为 true

```
if (yieldOnReturn) { // 如果定时器设备处理程序要求进行上下文切换，现在可以执行
 yieldOnReturn = FALSE;
 status = SystemMode; // 转为内核态
 currentThread->Yield(); // 切换到下一个线程
 status = old; // 恢复之前的状态
}
```

在处理中断时检查 yieldOnReturn 标志并调用 yield（）进行进程的切换

## Nachos 的文件系统（实验 4）

|                                                                                                                      |          |                  |
|----------------------------------------------------------------------------------------------------------------------|----------|------------------|
| 实验题目： 实验四 Nachos 的文件系统                                                                                               |          | 学号： 202100130176 |
| 日期： 2024. 4. 18                                                                                                      | 班级： 计科一班 | 姓名： 王宁波          |
| Email： 2197686837@qq.com                                                                                             |          |                  |
| <p>实验目的：</p> <p>(1) 理解 Nachos 硬盘是如何创建的；</p> <p>(2) 理解硬盘初始化的过程（如何在硬盘上创建一个文件系统）；</p> <p>(3) 了解 Nachos 文件系统在磁盘上的布局；</p> |          |                  |





```
wmb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ hexdump -c DISK
00000000 253 211 g E 200 \0 \0 \0 001 \0 \0 \0 002 \0 \0 \0
00000010 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
*
00000080 \0 \0 \0 \0 310 \0 \0 \0 002 \0 \0 \0 003 \0 \0 \0
00000090 004 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
000000a0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
*
00000100 \0 \0 \0 \0 037 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
00000110 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
*
00200000 \0 \0 \0 \0
00200004
```

hexdump -C DISK

```
wmb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ hexdump -C DISK
00000000 ab 89 67 45 80 00 00 00 01 00 00 00 02 00 00 00 |..gE.....|
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000080 00 00 00 00 c8 00 00 00 02 00 00 00 03 00 00 00 |.....|
00000090 04 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000000a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000100 00 00 00 00 1f 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00200000 00 00 00 00 |....|
00200004
```

2. `../lab5/main.cc` 调用了 `../threads/system.cc` 中的 `Initialize()` 创建了 Nachos 硬盘 DISK

```
#ifdef FILESYS
// 创建硬盘
 synchDisk = new SynchDisk("DISK");
#endif

#ifdef FILESYS_NEEDED
 fileSystem = new FileSystem(format);
#endif // 创建文件系统

#ifdef NETWORK
 postOffice = new PostOffice(netname, rely, order, 10);
#endif
```

- 分析 `../filesys/ synchdisk.cc` 及 `../machine/disk.cc`，理解 Nachos 创建硬盘的过程与方法；

```
//-----
// SynchDisk::SynchDisk
// 初始化与物理磁盘的同步接口，同时初始化物理磁盘。
// "name" -- 用作磁盘数据存储的UNIX文件名（通常为"DISK"）
//-----

SynchDisk::SynchDisk(char* name)
{
 // 初始化为0
 semaphore = new Semaphore("synch disk", 0); /*同步接口：信号量（Semaphore）和锁（Lock）来同步对磁盘的访问*/
 lock = new Lock("synch disk lock");
 disk = new Disk(name, DiskRequestDone, (_int) this); //创建磁盘对象
 //Disk 类是一个模拟磁盘的类，它提供了对磁盘的基本操作，如读写等。
}
```

- 命令 `nachos -f` 创建并格式化 Nachos 硬盘

```

Disk::Disk(char* name, VoidFunctionPtr callWhenDone, _int callArg)
{
 int magicNum;
 int tmp = 0;

 DEBUG('d', "Initializing the disk, 0x%x 0x%x\n", callWhenDone, callArg);
 handler = callWhenDone;
 handlerArg = callArg;
 lastSector = 0;
 bufferInit = 0;

 fileNo = OpenForReadWrite(name, FALSE); //尝试以读写模式打开名为 name 的文件
 if (fileNo >= 0) { //如果文件已存在则读取文件的魔数 (magic number) 并检查它是否等于预定义的 MagicNumber.
 //魔数是一个特殊的值, 用于标识文件或数据的类型。
 Read(fileNo, (char *) &magicNum, MagicSize);
 ASSERT(magicNum == MagicNumber);
 } else { // file doesn't exist, create it
 fileNo = OpenForWrite(name);
 magicNum = MagicNumber;
 WriteFile(fileNo, (char *) &magicNum, MagicSize); // write magic number
 /* 否则读取文件的魔数 (magic number) 并检查它是否等于预定义的 MagicNumber.
 魔数是一个特殊的值, 用于标识文件或数据的类型。 */
 Lseek(fileNo, DiskSize - sizeof(int), 0);
 WriteFile(fileNo, (char *) &tmp, sizeof(int));
 }
 active = FALSE; //表示当前没有磁盘请求正在处理
}

```

### 磁盘类的构造函数

- 分析 ../filesystems/filessys.cc , 特别是构造函数 FileSystem::FileSystem(..), 理解 Nachos 硬盘”DISK”的创建方法, 及硬盘格式化 (创建文件系统) 的处理过程;

在初始化文件系统时 (FileSystem 类的构造函数), 将这两个特殊的扇区置位 (已使用), 然后将硬盘位示图文件头与目录表文件头写入到这两个特殊的扇区中;

对于一个真正的操作系统, 由于系统启动时需要根据目录文件的文件头访问根目录, 因此为这两个特殊的结构分配到 0 号扇区与 1 号扇区这两个特殊的扇区中, 是为了便于系统启动时从已知的、固定的位置访问它们。

```

//-----
// FileSystem::FileSystem
// 初始化文件系统。如果 format = TRUE, 则磁盘上没有任何内容,
// 我们需要初始化磁盘以包含一个空目录和一个空闲扇区的位图
// (几乎所有扇区都标记为空闲)。
//
// 如果 format = FALSE, 则只需打开表示位图和目录的文件。
//
// "format" - 我们是否应该初始化磁盘?
//-----

FileSystem::FileSystem(bool format)
{
 DEBUG('f', "Initializing the file system.\n");
 // printf("sizeof(DirectoryEntry)=%d\n", sizeof(DirectoryEntry));
 // DirectoryEntry 整个结构体大小为 20Bytes

 if (format) {
 BitMap *freeMap = new BitMap(NumSectors);
 Directory *directory = new Directory(NumDirEntries);
 FileHeader *mapHdr = new FileHeader;
 FileHeader *dirHdr = new FileHeader;

 DEBUG('f', "Formatting the file system.\n");

 // 首先, 为目录和位图的文件头分配空间
 // (确保没有其他人占用这些空间!)
 freeMap->Mark(FreeMapSector);
 freeMap->Mark(DirectorySector);

 // 其次, 为包含目录和位图文件内容的数据块分配空间。
 }
}

```

- 分析 ../lab5/main.cc, 了解 Nachos 文件系统目前提供了哪些文件操作命令, 对每个命令进行测试, 根据执行结果观察哪些命令已经实现 (正

确运行), 哪些无法正确运行 (尚未完全实现, 需要你自己完善)

- -cp, -ap, -hap, -nap, 以及-p, -r, -l, -D, -t

**nachos [-d f] -cp UNIX\_filename nachos\_filename:** 将一个 Unix 文件系统中文件 UNIX\_filename 复制到 Nachos 文件系统中, 重新命名为 nachos\_filename;

该命令调用了 code/filesys/fstest.cc 中的函数 void Copy(char \*from, char \*to), 已经可以正常实现文件的复制功能 (Unix file→Nachos file)。

目前实现的 Nachos 文件系统尚未提供 creat()系统调用 (实验 6、7、8 中实现), 也就没有提供创建文件的命令。如果要在 Nachos 的硬盘中创建文件, 目前只能通过该命令从你的 UNIX 系统中复制一个文件到 Nachos 硬盘中;

**nachos [-d f] -p nachos\_filename:** 该命令输出 nachos 文件 nachos\_filename 的内容, 类似于 UNIX 中的 cat 命令;

**nachos [-d f] -r nachos\_filename:** 删除 Nachos 文件 nachos\_filename, 类似于 UNIX 中的 rm 命令;

**nachos [-d f] -l:** 输出当前目录中的文件名 (类似于 DOS 中的 dir, UNIX 中的 ls);

**nachos [-d f] -t:** 测试 Nachos 文件系统的性能 (目前尚未实现);

**nachos [-d f] -D:** 输出 Nachos 的文件系统在磁盘上的组织。打印出整个文件系统的所有内容, 包括位图文件 (bitmap)、文件头 (file header)、目录文件 (directory) 和普通文件 (file)

5. 分析 ../lab5/fstest.cc 及 ../filesys/filessys.cc, 理解 Nachos 对这些命令的处理过程与方法; 思考如何完善尚未完成的命令 (如 -ap, -hap, -nap );

在 main 函数中对参数进行处理并调用相应的函数, 其中-f 是在 Initialize 初始化内核时进行处理, 使 format 的值为真, 表示初始化文件系统, 硬盘 DISK 和文件系统都是在 Initialize 函数中完成的

6. 利用命令 hexdump -C DISK 查看硬盘格式化后硬盘的布局, 理解格式化硬盘所完成的工作, 以及文件系统管理涉及到的一些数据结构组织与使用, 如位示图文件头, 文件头 (FCB)、目录表与目录项、空闲块管理位示图等;

```
wnb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ hexdump -C DISK
00000000 ab 89 67 45 80 00 00 00 01 00 00 00 02 00 00 00 |..gE.....|
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000080 00 00 00 00 c8 00 00 00 02 00 00 00 03 00 00 00 |.....|
00000090 04 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000000a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000100 00 00 00 00 1f 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00020000 00 00 00 00 |....|
00020004
```



```
wmb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ od -c test/small
0000000 T h i s i s t h e s p r i
0000020 n g o f o u r d i s c o n
0000040 t e n t . \n
0000046
wmb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ hexdump -c test/small
0000000 T h i s i s t h e s p r i
0000010 n g o f o u r d i s c o n
0000020 t e n t . \n
0000026
wmb@wnb-virtual-machine:~/OS-MAIN/nachos-3.4-SDU/code/filesys$ hexdump -C test/small
00000000 54 68 69 73 20 69 73 20 74 68 65 20 73 70 72 69 |This is the spri|
00000010 6e 67 20 6f 66 20 6f 75 72 20 64 69 73 63 6f 6e |ng of our discon|
00000020 74 65 6e 74 2e 0a |tent..|
00000026
```

- 利用命令 `hexdump -C DISK` 查看硬盘格式化后硬盘的布局，理解创建一个文件后相关的结构在硬盘上的存储布局；

表 5-3 Nachos 的磁盘详细布局

| 扇区号 | 起止字节        | 内容描述                | 内容               | 大小     |
|-----|-------------|---------------------|------------------|--------|
|     | 0x0~0x3     | 魔数：磁盘标识(文件系统标识)     | 0x456789ab       | 4 字节   |
| 0   | 0x4~0x83    | 位示图文件头              | Class FileHeader | 128 字节 |
| 1   | 0x84~0x103  | 目录表文件头              | Class FileHeader | 128 字节 |
| 2   | 0x104~0x183 | 位示图文件数据块            | Class BitMap     | 128 字节 |
| 3   | 0x184~0x203 | 根目录表                | Class Directory  | 128 字节 |
| 4   | 0x204~0x283 | 根目录表                | Class Directory  | 128 字节 |
| 5   | 0x284~0x303 | 第一个文件的文件头           | Class FileHeader | 128 字节 |
| 6   | 0x304~0x383 | 第一个文件的数据块           |                  | 128 字节 |
| 7   |             | (文件需要的块数不定)         | .....            |        |
|     |             | 第二个文件的文件头           | Class FileHeader | 128 字节 |
|     |             | 第二个文件的数据块           |                  | 128 字节 |
|     |             | .....               | .....            |        |
|     |             | 第三个文件的文件头           | Class FileHeader | 128 字节 |
|     |             | 第三个文件的数据块           |                  | 128 字节 |
|     |             | .....               | .....            |        |
|     |             | 第一个文件的数据块（扩展第一个文件时） |                  | 128 字节 |
|     |             | 以此类推                |                  |        |
|     |             |                     |                  |        |

- 复制更多的文件到 DISK 中，然后利用命令 “`nachos -r 文件名`” 删除一个文件，利用 `hexdump -C DISK` 查看文件的布局，分析文件系统的管理策略。  
依次复制了 `small`，`mideum` 和 `big` 后运行 `nachos -D`

### 删除 small 之后

在位示图中清除 samll 文件数据块所占用 6 号扇区，和 samll 文件头所占用的 5 号扇区；清除目录表中为 samll 文件所分配的目录项中的 inUse 位（3 号扇区中 0x184 内容 1 变为 0，即位 small 文件分配的目录项变为空闲）

因此删除一个文件后，该文件在目录表中文件名、文件头所占的扇区号均未清除，只是将该目录项变为空闲；文件头中的信息（文件大小、文件所占用的扇区数以及为文件数据分配的扇区列表）也未清除；文件的内容也未清除；



[illegible]

### 结论分析与体会:

通过本次实验，我对 Nachos 操作系统中的文件系统有了更深入的理解，尤其是文件系统的创建、管理和命令的执行机制。以下是我的一些主要收获和体会：

## 1. 硬盘的初始化与文件系统的创建

- 硬盘创建：通过分析 `disk.cc` 和 `synchdisk.cc`，我理解了 Nachos 虚拟硬盘的创建过程，这涉及到硬盘的底层数据结构以及如何通过同步机制处理多线程环境下的磁盘访问。
- 文件系统的格式化：在 `FileSystem::FileSystem(..)` 构造函数中，我详细了解了文件系统在硬盘上的格式化过程，包括如何初始化位示图（空闲块管理）、目录表以及文件头。

## 2. 文件系统命令的实现与测试

- 命令测试：通过实际测试已实现的命令（如`-cp`, `-p`, `-r`, `-l`, `-D`, `-t`等），我观察到这些命令如何与文件系统交互，例如文件的复制、读取、删除和列表显示。
- 命令完善：对于尚未完善的命令（`-ap`, `-hap`, `-nap`），我通过分析它们应该如何与现有文件系统框架下实现，思考了改进策略，如对文件进行追加写入而非覆盖。

### 3. 文件系统结构的分析

- 硬盘布局：通过 hexdump 命令查看的硬盘布局让我具体了解了文件系统中的数据结构如何在硬盘上进行存储和管理。
- 文件和目录管理：通过分析文件头（FCB）、目录表的实现，我加深了对文件数据块分配、目录项的管理及其在硬盘上的存储布局的理解。

#### 4. 文件删除与硬盘空间管理

- 文件删除策略：删除文件时，Nachos 文件系统并未清除文件头和文件内容，而仅将目录项标记为未使用。

## 5. 多级目录的设计思路

- 目录结构的扩展：尽管 Nachos 默认仅支持单级目录，我考虑到通过树形结构实现多级目录。这需要修改目录表的设计，以支持目录项可以指向另一个目录

表，从而形成目录的层级结构。

体会：通过本次实验，我不仅提高了对操作系统文件管理核心概念的理解，也锻炼了我的实践能力和问题解决能力。对文件系统的深入研究有助于我在未来的学习和工作中更好地理解现代操作系统的复杂机制。这次实验也让我意识到理论与实践的结合对于深入掌握复杂系统的重要性。

## 扩展 Nachos 的文件系统（实验 5）

|                                                                                                                                                                                                                                                                                                 |          |                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|
| 实验题目： 实验五 扩展 Nachos 的文件系统                                                                                                                                                                                                                                                                       |          | 学号： 202100130176 |
| 日期： 2024. 4. 19                                                                                                                                                                                                                                                                                 | 班级： 计科一班 | 姓名： 王宁波          |
| Email： 2197686837@qq. com                                                                                                                                                                                                                                                                       |          |                  |
| <p>实验目的：</p> <ol style="list-style-type: none"><li>1. 理解文件系统中文件操作的实现方法，如文件打开、读、写、扩展、定位、关闭等操作的实现机理；</li><li>2. 理解如何管理硬盘空闲块；</li><li>3. 创建文件时，如何为文件分配目录项及文件头（FCB）；</li><li>4. 理解文件扩展时，如何为要扩展的数据查找并分配空闲块；</li><li>5. 理解文件扩展后，文件大小是如何记录与保存的；</li><li>6. 文件被删除后，如何回收为其分配的资源，如文件头、目录项、硬盘块等；</li></ol> |          |                  |
| <p>实验要求：</p> <p>目前 Nachos 实现的文件系统存在诸多限制，其中之一是文件大小不能扩展，即无法在已经存在的文件尾部追加数据；</p>                                                                                                                                                                                                                    |          |                  |



该实验的任务就是让你修改 Nachos 的文件系统，以满足：

- (1) 文件创建时，其大小可初始化为 0；
- (2) 当一个文件写入更多的数据时，其大小可随之增大；
- (3) 要求能够在从一个文件的任何位置开始写入数据，即能够正确处理命令 `-ap`, `-hap`, 及 `-nap`; (`-ncp`)

需要理解 `../code/machine` 及 `../code/filesys` 目录下的相关源代码 (`class Disk`、`class SynchDisk`、`class BitMap`、`class FileHeader`、`class OpenFile`、`class Directory` 及 `class FileSystem` 等) 以及 `../lab5` 下的两个模块 (`main.cc` 及 `fstest.cc`)，分析 Nachos 不能对文件进行扩展的原因；

## 实验步骤与内容：

### 1、修改 `OpenFile::WriteAt()`

```
147 OpenFile::WriteAt(char *from, int numBytes, int position)
148 { // 拓展为可以从文件任何位置开始写入数据
149 int fileLength = hdr->FileLength(); // 被写入文件的长度
150 int i, firstSector, lastSector, numSectors;
151 bool firstAligned, lastAligned;
152 char *buf;
153
154 // 约束1. 写入的字节数小于等于0或者写入的位置大于文件长度，返回-1
155 if (numBytes <= 0 || (position > fileLength)) {
156 return -1;
157 }
158 // 约束2. 写入的位置加上写入的字节数大于文件长度，需要进行拓展
159 // printf("u:%d + %d > %d\n", position, numBytes, fileLength);
160 if ((position + numBytes) > fileLength) { // 证明文件需要进行拓展
161 int incrementBytes = (position + numBytes) - fileLength; // 拓展的字节数
162 BitMap *freeBitMap = fileSystem->getBitMap(); // 获取位示图
163 // printf("u:check bitmap\n");
164 // freeBitMap->Print();
165 bool hdrRet;
166 // 输入参数: 位示图, 当前文件长度, 增加字节数
167 hdrRet = hdr->Allocate(freeBitMap, fileLength, incrementBytes); // 分配磁盘块
168 if (!hdrRet) {
169 // printf("constraint 2\n");
170 return -1;
171 }
172 fileSystem->setBitMap(freeBitMap); // 更新位示图
173 }
```

只需修改两个不能对文件进行扩展的两个约束即可，后续代码保持不变；

### 2、修改 `FileSystem` 类，增加 `setBitMap()` 与 `getBitMap()`

**功能：从硬盘读取空闲块位示图文件，内容被修改后再将其写回硬盘**

类 `FileSystem` 在其构造函数中，维护了两个一直处于打开状态的文件句柄 `freeMapFile` 与 `directoryFile`，通过 `freeMapFile` 读写位示图。

```
BitMap*
FileSystem::getBitMap() {
 // numSector: DISK上总扇区数 (共有32*32=1024个扇区)
 BitMap *freeBitMap = new BitMap(NumSectors);
 freeBitMap->FetchFrom(freeMapFile); // 从磁盘上读取位示图 freeMapFile = new OpenFile(FreeMapSector)
 return freeBitMap;
}

void
FileSystem::setBitMap(BitMap* freeMap) {
 freeMap->WriteBack(freeMapFile);
}
```

### 3、修改 `OpenFile::OpenFile()` 及 `OpenFile::WriteBack()`

实现文件头的硬盘读写

```

OpenFile::OpenFile(int sector)
{
 hdr = new FileHeader;
 hdr->FetchFrom(sector); // 从磁盘扇区sector中读取文件头的内容到hdr中
 seekPosition = 0; // 并将读写指针（偏移量）设置为开始位置（0）
 hdrSector = sector; // 记录文件的文件头所在扇区sector以便写回
}

```

```

void
OpenFile::WriteBack() {
 hdr-> WriteBack(hdrSector);
 // 拓展之后将文件头更新
}

```

#### 4、修改 FileHeader::Allocate()

目的：为要写入的文件数据分配硬盘空间，写入数据可能利用文件的最后一个扇区的剩余空间，也可能为其新分配扇区（硬盘块）；

- FileHeader 构造函数  
显式定义构造函数，防止无效内容

```

// 显式初始化
FileHeader::FileHeader() {
 numBytes = 0;
 numSectors = 0;
 for (int i = 0; i < NumDirect; i++) {
 dataSectors[i] = 0;
 }
}

```

- FileHeader::Allocate()  
判断是否需要为写入数据分配新的扇区，如果需要就为其分配，并更新位示图及文件头三元组



```
nachos -ap test/small | small
```

nachos -ap test/big small

nachos - ap test/medium medium

nachos - hap test/medium small

nachos - nap medium small

nachos - r small

反复运行nachos -ap test/big small

```
● dundee@ubuntu:~/Documents/nachos-3.4-SDU/code/lab5$./nachos -ap test/big small
```

- 修改了 `OpenFile::WriteAt()` 函数，去除了原有的不能对文件大小进行扩展的限制，实现了在文件任意位置写入数据的功能。

|                                                                                                                                                                                                                                                                          |  |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| <ul style="list-style-type: none"><li>在 <code>FileSystem</code> 类中添加了 <code>setBitMap()</code> 和 <code>getBitMap()</code> 方法，确保对空闲块位示图的更改能够正确读取和回写到硬盘，从而有效地管理磁盘空间。</li><li>通过修改 <code>FileHeader::Allocate()</code> 函数，实现了当文件需要更多空间时，能够动态分配硬盘块，并正确更新文件头信息和位示图。</li></ul> |  |  |
| 2. 理解文件系统的数据结构：                                                                                                                                                                                                                                                          |  |  |
| <ul style="list-style-type: none"><li>深入分析了 <code>FileHeader</code>、<code>OpenFile</code>、<code>Directory</code> 和 <code>BitMap</code> 等数据结构的设计和作用，这些结构是文件系统能够正确运行的基础。</li><li><code>FileHeader</code> 的修改尤为关键，它不仅需要记录文件的基本信息，如文件大小和存储位置，还需要在文件扩展时动态更新。</li></ul>        |  |  |
| 3. 测试与验证：                                                                                                                                                                                                                                                                |  |  |
| <ul style="list-style-type: none"><li>通过一系列的测试命令，如 <code>nachos -ap</code>，<code>nachos -hap</code>，和 <code>nachos -nap</code> 等，验证了修改后的文件系统能够正确处理文件的追加和位置指定追加操作。</li><li>测试结果表明，新的文件系统能够有效地在保持数据一致性的同时，动态扩展文件大小。</li></ul>                                              |  |  |
| 4. 实际应用与理论联系：                                                                                                                                                                                                                                                            |  |  |
| <ul style="list-style-type: none"><li>本实验加深了我对文件系统理论的理解，特别是如何在实际系统中实现理论中的文件操作和管理。</li><li>通过动手实践，我更清楚地看到了理论与实际应用之间的联系和区别，理解了为何实际系统设计需要考虑性能、可靠性和数据一致性等多方面因素。</li></ul>                                                                                                    |  |  |

## Nachos 用户程序与系统调用（实验 6）

|                            |          |                  |
|----------------------------|----------|------------------|
| 实验题目： 实验六 Nachos 用户程序与系统调用 |          | 学号： 202100130176 |
| 日期： 2024. 5. 10            | 班级： 计科一班 | 姓名： 王宁波          |
| Email： 2197686837@qq. com  |          |                  |
| 实验目的：                      |          |                  |

1. 为后续实验中实现系统调用 Exec() 与 Exit() 奠定基础
2. 理解 Nachos 可执行文件的格式与结构
3. 掌握 Nachos 应用程序的编程与法, 了解用户进程是如何通过系统调用与操作系统内核进行交互
4. 掌握如何利用交叉编译生成 Nachos 的可执行程序
5. 理解如何将用户线程映射到核心线程, 核心线程执行用户程序的原理与方法
6. 理解当前进程的页表是如何与 CPU 使用的页表进行关联的

### 实验要求:

1. 阅读 ../bin/noff.h, 分析 Nachos 可执行程序 .noff 文件的格式组成
2. 阅读 ../test 目录下的几个 Nachos 应用程序, 理解 Nachos 应用程序的编程语法, 了解用户进程是如何通过系统调用与操作系统内核进行交互的
3. 阅读 ../test/Makefile, 掌握如何利用交叉编译生成 Nachos 的可执行程序
4. 阅读 ../threads/main.cc, ../userprog/progtest.cc, 根据对命令行参数 -x 的处理过程, 理解系统如何为应用程序创建进程, 并启动进程的
5. 阅读 ../userprog/progtest.cc, ../threads/scheduler.cc (Run()), 理解如何将用户线程映射到核心线程, 以及核心线程执行用户程序的原理与方法
6. 阅读 ../userprog/progtest.cc, ../machine/translate.cc, 理解当前进程的页表是如何与 CPU 使用的页表进行关联的

### 实验步骤与内容:

1. 理解 Nachos 应用程序的编写与生成原理
- 1.1. Nachos 可执行文件.noff 文件的格式

```
typedef struct segment {
 int virtualAddr; /* location of segment in virt addr space */
 int inFileAddr; /* location of segment in this file */
 int size; /* size of segment */
} Segment;

typedef struct noffHeader {
 int noffMagic; /* should be NOFFMAGIC */
 Segment code; /* executable code segment */
 Segment initData; /* initialized data segment */
 Segment uninitData; /* uninitialized data segment --
 * should be zero'ed before use
 */
} NoffHeader;
```

段的结构体 Segment 记录了该段起始的虚拟地址、在文件中的地址以及段大小; noff 文件头结构体记录了该可执行文件的魔数、代码段、初始化数据段和未初始化数据段

AddrSpace::AddrSpace() 负责将程序装入主存, 并设置 PC 的值部分代码, 其构造函数在将 NOFF 文件装入内存之前, 先打开该文件, 读入文件头, 然后根据文件头信息确定程序所占用的空间, 将相应的段装入内存, 将程序的入口地址, 给 PC 赋值。

- 1.2. Nachos 应用程序的编程语法



Nachos 的应用程序采用类 C 语言语法，通过 Nachos 提供的系统调用进行编写，但是涉及到系统调用时，需要引入头文件 `syscall.h` 来调用系统调用，由此实现与系统内核的交互。

```
#include "syscall.h"
```

```
int main() {
 int i,j,k;
 k = 3;
 i = 2;
 j = i-1;
 k = i-j+k;
 Halt();
}
```

### 1. 3. 利用交叉编译生成 Nachos 可执行程序

Nachos 提供了一个交叉编译程序可将 Nachos 用户编写的应用程序编译成 MIPS 指令集的可执行程序，然后在 Nachos 中运行

其中 gcc MIPS 交叉编译器首先将 Nachos 的应用程序编译成 COFF 格式的可执行文件，然后利用 `../test/coff2noff` 将 COFF 格式的可执行程序转换成 Nachos CPU 可识别的 NOFF 可执行程序

## 2. 理解 Nachos 执行用户程序的原理

### 2. 1. Nachos 如何为应用程序创建并启动进程

Nachos 的参数 `-x` ( `nachos -x filename` ) 调用 `../userprog/progtest.cc` 的 `StartProcess(char *filename)` 函数，为用户程序创建 `filename` 创建相应的进程，并启动该进程的执行。

```
void
StartProcess(char *filename)
{
 OpenFile *executable = fileSystem->Open(filename); // 打开文件
 AddrSpace *space; // 定义地址空间

 if (executable == NULL) {
 printf("Unable to open file %s\n", filename);
 return;
 }
 space = new AddrSpace(executable); // 为应用程序分配内存地址空间
 currentThread->space = space; // 将该进程映射到一个核心线程
 space->Print(); // 输出该作业的页表信息
 delete executable; // 关闭文件

 space->InitRegisters(); // 初始化 CPU 的寄存器，包括数据寄存器、PC 以及栈指针 set the initial register values
 space->RestoreState(); // 是将用户进程的页表传递给系统核心 (Machine 类) load page table register

 machine->Run(); // 从程序入口开始，完成取指令、译码、执行的过程 jump to the user program
 ASSERT(FALSE); // machine->Run never returns;
 // the address space exits
 // by doing the syscall "exit"
}
```

### 2. 2. 如何将用户线程映射到核心线程以及如何执行用户程序的原理与方法



```

#ifdef USER_PROGRAM // ignore until running user programs
// 1. 将当前CPU寄存器的内容保存到旧进程的用户寄存器中
// 2. 保存用户页表
if (currentThread->space != NULL) { // if this thread is a user program,
 currentThread->SaveUserState(); // save the user's CPU registers
 currentThread->space->SaveState();
}
#endif

```

```

#ifdef USER_PROGRAM
// 1. 如果运行用户进程，需要将当前进程的用户寄存器内存加载到CPU寄存器中
// 2. 并且加载用户页表
if (currentThread->space != NULL) { // if there is an address space
 currentThread->RestoreUserState(); // to restore, do it.
 currentThread->space->RestoreState();
}
#endif

```

上图为../threads/scheduler.cc 中的 Scheduler::Run() 函数的部分截图, 其中涉及了在线程切换时如何处理当前运行的用户程序相关状态的算法。

当运行用户程序时, 宏定义 USER\_PROGRAM 生效, 内部代码会被编译。这两段代码发生在该函数的开头与结尾处, 通过判断当前核心线程是否映射了地址空间来判断是否有用户线程映射到了核心线程。如果有用户线程映射到了核心线程, 则会在上下文切换之前保存当前线程的寄存器状态, 以及在上下文切换结束后恢复新线程的寄存器状态到核心线程。

### 3. 理解当前进程的页表是如何与 CPU 使用的页表进行关联的

为使该核心线程能够执行用户进程的代码, 需要将用户页表首地址传递给核心的地址变换机构; (machine.h 中维护一个 pageTable 指针, 指向当前正在运行的 Nachos 应用进程的页表)

AddrSpace::RestoreState() 将用户进程的页表传递给 Machine 类, 该页表在为用户进程分配地址空间时创建

```

void AddrSpace::RestoreState()
{
 // 除以下赋值操作外没有显式对页表的操作
 machine->pageTable = pageTable;
 machine->pageTableSize = numPages;
}

```

为类 AddrSpace 新增一个成员函数 AddrSpace::Print() 输出页表信息, 包括分配的页表和页与页框的映射关系。

```

void AddrSpace::Print() {
 printf("userprog: %d, page table dump: %d pages in total\n", pid, numPages);
 printf("=====\n");
 printf("\tVirtPage\tPhysPage\n");
 for (int i=0; i < numPages; i++) {
 printf("\t%d\t\t%d\n", pageTable[i].virtualPage, pageTable[i].physicalPage);
 }
 printf("=====\n\n");
}

```

### 结论分析与体会：

通过实验六我学习了 Nachos 中 .noff 文件的格式，如何创建用户程序以及如何运行用户程序，理解页表的创建映射和地址变换的过程，这加深了我对 nachos 系统的理解，对我接下来开展实验 7、8 有很大帮助

## 地址空间的扩展（实验 7）

|                   |          |                  |
|-------------------|----------|------------------|
| 实验题目： 实验七 地址空间的扩展 |          | 学号： 202100130176 |
| 日期： 2024. 5. 12   | 班级： 计科一班 | 姓名： 王宁波          |

## 实验目的:

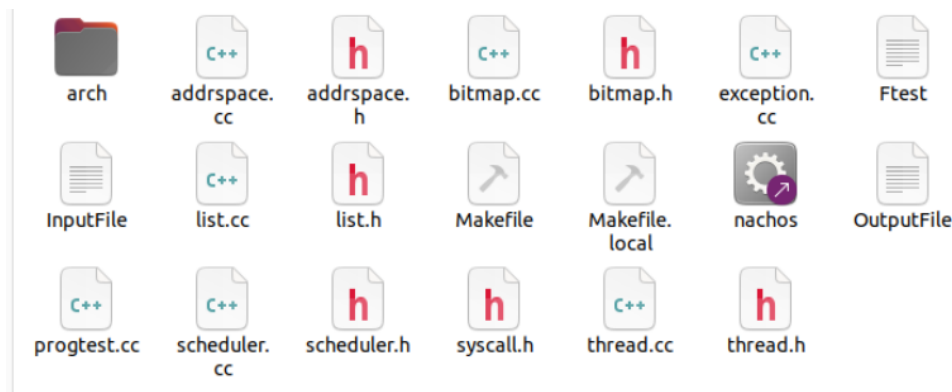
1. 通过考察系统加载应用程序过程, 如何为其分配内存空间、创建页表并建立虚页与实页帧的映射关系, 理解 Nachos 的内存管理方法。
2. 理解 Nachos 系统如何对空闲帧进行管理。
3. 理解如何加载另一个应用程序并为其分配地址空间, 一直吃多进程机制。
4. 理解进程的 pid。
5. 理解进程退出所要完成的工作。

## 实验要求:

1. 阅读 ../prog/protest.cc, 深入理解 Nachos 创建应用进程的详细过程
2. 阅读类 AddrSpace, 然后对其进行修改, 使 Nachos 能够支持多进程机制, 允许 Nachos 同时运行多个用户进程。
3. 在类 AddrSpace 中添加完善 Print() 函数。
4. 在类 AddrSpace 中实例化类 Bitmap 的一个全局对象, 用于管理空闲帧。
5. 如果将 SpaceId 直接作为进程号 pid 是否合适? 如果感觉不是很合适, 应该如何为进程分配相应的 pid。
6. 为实现 Join(pid), 考虑如何在该进程相关联的核心进程中保存进程号。
7. 根据进程创建时系统为其所做的工作, 考虑进程退出时应该做哪些工作。
8. 考虑系统调用 Exec() 和 Exit() 的设计实施方案。

## 实验步骤与内容:

1. 创建实验目录  
参照实验 2, 包含以下文件并对 Makefile、Makefile.local 做出修改



2. 拓展地址空间以增加实现多进程机制
- 2.1. 分析

```
// first, set up the translation
pageTable = new TranslationEntry[numPages];
for (i = 0; i < numPages; i++) {
 pageTable[i].virtualPage = i; // for now, virtual page # = phys page #
 pageTable[i].physicalPage = i;
 pageTable[i].valid = TRUE;
 pageTable[i].use = FALSE;
 pageTable[i].dirty = FALSE;
 pageTable[i].readOnly = FALSE; // if the code segment was entirely on
 // a separate page, we could set its
 // pages to be read-only
}
```

根据 AddrSpace 的构造函数可知，目前实现的 Nachos 系统，当加载一个应用程序并为其分配内存时，总是将程序的第 0 号页面分配到内存的第 0 号帧，第 1 号页面分配到内存的第 1 号帧，…，以此类推。

因此当 Nachos 应用程序中通过系统调用 Exec() 装入并执行另一个 Nachos 应用程序 ./test/exec.noff 时，会覆盖该进程本身的内存空间，因此要做出修改。

## 2.2. 修改物理页的分配机制

我们可以利用位示图对内存的空闲帧进行管理，将原本的直接分配改为查找空闲的物理帧。

```
static BitMap *freeMap = new BitMap(128); //用于管理空闲帧
```

```
pageTable = new TranslationEntry[numPages];
for (i = 0; i < numPages; i++) { // 虚页实页映射
 pageTable[i].virtualPage = i; // for now, virtual page # = phys page #
 pageTable[i].physicalPage = freeMap->Find(); // 寻找空闲页进行分配
 pageTable[i].valid = TRUE;
 pageTable[i].use = FALSE;
 pageTable[i].dirty = FALSE;
 pageTable[i].readOnly = FALSE; // if the code segment was entirely on
 // a separate page, we could set its
 // pages to be read-only
}
```

## 2.3. 实现虚拟页到物理页的地址变换

主要是实现虚拟页到物理页的地址变换，而不是直接映射，首先需要去掉初始化内存的语句，在通过虚拟地址和页大小计算出所在页 vpn 和页内偏移量 offset，然后得出所在页对应的物理帧

```
// then, copy in the code and data segments into memory
if (noFFH.code.size > 0) {
 DEBUG('a', "Initializing code segment, at 0x%x, size %d\n",
 noFFH.code.virtualAddr, noFFH.code.size);
 int virtualAddr = noFFH.code.virtualAddr;
 int vpn = (unsigned) virtualAddr / PageSize; // 页号
 int offset = (unsigned) virtualAddr % PageSize; // 页内偏移
 int physicalAddr = pageTable[vpn].physicalPage * PageSize + offset; // 物理地址
 executable->ReadAt(&(machine->mainMemory[physicalAddr]),
 noFFH.code.size, noFFH.code.inFileAddr);
}
if (noFFH.initData.size > 0) {
 DEBUG('a', "Initializing data segment, at 0x%x, size %d\n",
 noFFH.initData.virtualAddr, noFFH.initData.size);
 int virtualAddr = noFFH.initData.virtualAddr;
 int vpn = (unsigned) virtualAddr / PageSize; // 页号
 int offset = (unsigned) virtualAddr % PageSize; // 页内偏移
 int physicalAddr = pageTable[vpn].physicalPage * PageSize + offset; // 物理地址
 executable->ReadAt(&(machine->mainMemory[physicalAddr]),
 noFFH.initData.size, noFFH.initData.inFileAddr);
}
ref = 0;
// Print();
}
```

## 3. 系统调用 Exec() 和 Exit() 的设计实现

### 3. 1. 理解 Nachos 系统调用的实现原理

#### 3. 1. 1. 系统调用的入口

交叉编译应用程序 exe.c，汇编代码如下

```
.ascii "../test/halt.noff\000" # 参数地址
.text
.align 2
.globl main
.ent main

.frame 定义了访问局部栈帧寄存器 $fp、栈帧大小 24 以及返回地址 $31 .frame $fp,24,$31 # vars= 0, regs= 2/0, args= 16, ext
.mask 0xc0000000,-4
.fmask 0x00000000,0
subu $sp,$sp,24 # 移动栈指针$sp,开辟大小 24 的栈帧
sw $31,20($sp) # 将返回地址保存 $31->$sp+20
sw $fp,16($sp) # 将原帧指针保存 $fp->$sp+16
move $fp,$sp # 移动帧指针
jal __main # 执行 main 函数
la $4,$LC0 # 将 Exec 的参数地址传给 4 号寄存器 jal Exec # 跳转至 Exec 函数,入口在 start.s 中

定义
jal 跳转时会保存返回地址即 PC+4->$31
jal Halt # 跳转至 Halt 函数,入口在 start.s 中 定义 PC+4->$31
$Li:
move $sp,$fp # 恢复栈指针
lw $31,20($sp) # 恢复返回地址 $sp+20->$31
lw $fp,16($sp) # 恢复帧指针 $sp+16->$fp
addu $sp,$sp,24 # 释放栈帧
j $31 # main 函数返回
.end main
```

由上述的汇编代码可以看出，在执行系统调用之前，代码 `la $4, $LC0` 首先将系统调用 `Exec()` 所需要的参数地址保存在 4 号寄存器中

查看 `start.s` 可知，该汇编文件保存了系统调用的入口，在执行系统调用时，跳转到入口地址，将系统调用号保存到 2 号寄存器中，然后执行系统调用指令。

```
.globl Halt
.ent Halt
Halt:
 addiu $2,$0,SC_Halt //system call code in r2
 syscall
 j $31
.end Halt

.globl Exit
.ent Exit
```

`progtest.cc` 中 `machine->Run()` 调用 `Machine::OneInstruction()` 处理指令时，如果当前模拟的指令为 `syscall`，那么会抛出一个 `SyscallException` 的异常，然后转而调用函数 `RaiseException()` 进入到内核态调用 `ExceptionHandler()` 具体执行

```
void
Machine::RaiseException(ExceptionType which, int badVAddr)
{
 DEBUG('m', "Exception: %s\n", exceptionNames[which]);
 // ASSERT(interrupt->getStatus() == UserMode);
 registers[BadVAddrReg] = badVAddr;
 DelayedLoad(0, 0); // finish anything in progress
 interrupt->setStatus(SystemMode);
 ExceptionHandler(which); // interrupts are enabled at this point
 //see userprog/exception.cc
 interrupt->setStatus(UserMode);
}
```

在具体执行时，先从 2 号寄存器中找到调用号，并根据系统调用号执行相应的函数，在实现系统调用时，需要在这里添加 case 选项

```
void
ExceptionHandler(ExceptionType which)
{
 int type = machine->ReadRegister(2);

 if ((which == SyscallException) && (type == SC_Halt)) {
 DEBUG('a', "Shutdown, initiated by user program.\n");
 interrupt->Halt();
 } else {
 printf("Unexpected user mode exception %d %d\n", which, type);
 ASSERT(FALSE);
 }
}
```

3.1.2. 系统调用 Exec() 与 Exit() 的设计实施方案  
首先在 ./userpog/excption.c 中增加对 SC\_Exec 的处理，调用 Machine:ReadRegister(4) 函数，从四号寄存器中取出参数地址 addr，然后读取参数，最后调用 StarPoces(filename)，装入并执行应用程序

结论分析与体会：  
实验七要求我们使用位示图来对地址空间进行扩展，改变了原有的固定分配的方式，而是使用位示图来记录空闲页并分配之，进而使得 Nachos 支持多道程序运行。  
在分配页面时，首先需要判断当前是否有充足的页面；当加载代码段和数据段的内容至内存时，需要将虚拟地址装换为物理地址。在释放地址空间时，需要在位示图中释放相应的页面，避免资源的浪费。

# 系统调用 Exec()与 Exit()（实验 8）

|                                |          |                  |
|--------------------------------|----------|------------------|
| 实验题目： 实验八 系统调用 Exec() 与 Exit() |          | 学号： 202100130176 |
| 日期： 2024. 5. 21                | 班级： 计科一班 | 姓名： 王宁波          |
| Email： 2197686837@qq. com      |          |                  |
| 实验目的：                          |          |                  |

1. 理解用户进程是如何通过系统调用与操作系统内核进行交互的。
2. 理解系统调用是如何实现的。
3. 理解系统调用参数传递与返回数据的回传机制。
4. 理解核心进程如何调度执行应用程序进程。
5. 理解进程退出后如何释放内存等为其分配的资源。
6. 理解进程号 pid 的含义与作用。

### 实验要求:

1. 阅读 ../userprog/exception.cc , 理解系统调用 Halt() 的实现原理。
2. 基于实验 6、7 中完成的工作, 利用 Nachos 提供的文件管理、内存管理及线程管理等功能, 编程实现系统调用 Exec() 和 Exit() 等。

### 实验步骤与内容:

#### 1. 实验准备

##### 1.1. 用户进程通过系统调用与操作系统内核进行交互的方式

每一条用户程序中的指令在虚拟机中被读取后, 被包装成一个 OneInstruction 对象, 用户进程映射到核心线程之后运行 machine::run() 时, 调用的 OneInstruction(Instruction \*instr) 函数检测到该条指令要调用系统调用 (case OP\_SYSCALL:), 则抛出异常 SyscallException, 进入核心态之后由 ExceptionHandler 去处理具体的系统调用。

##### 1.2. 系统调用是如何实现的

当汇编代码中出现类似 jal Exec 的代码时, 跳转到 start.s 的系统调用入口, 在其中保存参数的地址到相关的寄存器中, 并保存特定的系统调用号, 然后执行 syscall

##### 1.3. 系统调用参数传递与返回数据的回传机制

###### (1) 关于系统调用的寄存器的介绍:

- \$2: 用于存放系统调用类型, 以及系统调用的返回值;
- \$4: 存放系统调用的第一个参数;
- \$5: 存放系统调用的第二个参数;
- \$6: 存放系统调用的第三个参数;
- \$7: 存放系统调用的第三个参数;

###### (2) 参数传递过程

对于 Nachos 的系统调用, 一般是将要传递的参数依次保存到寄存器 \$4-7 中, 参数多于 4 个时, 其余的利用堆栈进行传递, 然后根据这些寄存器中的地址从内存中读出相应的参数, 当字符串作为参数时的传递方式时, 寄存器中保存的是字符串在内存中的地址

###### (3) 用于传递的函数

Machine::ReadRegister(intum) 可读取寄存器 num 中的内容, 得到内存地址 adr 或立即数;

Machine::ReadMem(int adr, int size, int \*value) 从内存 adr 处读取 size 个字节的内容存放到 value 所指向的单元中;

Machine::WriteRegister(intum, int value) 将寄存器 num 的值置为

value, 可以返回值写入寄存器\$2 中;

#### 1.4. 进程退出后如何释放内存等为其分配的资源

需要通过就绪队列与所有的等待队列以确定一个线程是否已经退出;就绪队列只有一个, 比较容易检查, 但等待队列可能有多个(每个信号量都对应一个, 且信号量的个数未知);为线程增加一个 TERMINATED 状态, 相应地增加一个 terminated 队列, 将所有的线程在调用 Finish() 后先进入该队列, 再伺机销毁

#### 1.5. 理解进程号 pid 的含义与作用

Exec() 返回类型为 SpaceId 的值, 该值将来作为系统调用 Join() 的参数以识别新建的用户进程, 可以在内核中如何记录它, 以便 Join() 能够通过该值找到对应的线程

## 2. 系统调用的实现

#### 2.1. 实现情况

包括基础系统调用 Exec()、Exit()、Join()、Fork()、Yield()、以及文件相关的系统调用 Open()、Creat()、Close()、Write()、Read()

#### 2.2. 实现 SpaceId

借鉴 UNIX 的思想, 我们可以为一个地址空间或该地址空间对应的进程分配一个唯一的整数, 例如 0~99 预留为核心进程使用, 用户进程号从 100 开始使用, 到 255 结束。

为类 AddrSpace 增添一个私有成员变量 SpaceId pid 和一个公共成员函数 SpaceId getSpaceId() { return pid; }

```
SpaceId GetSpaceId() { return pid; }

int ref;

private:
 TranslationEntry *pageTable; // Assume linear page table translation
 // for now!
 unsigned int numPages; // Number of pages in the virtual
 // address space
 SpaceId pid;
};
```

为 SpaceId 建立一个指针数组 addrspaces 保存 pid 和对应的地址空间的对应关系, 其中 addrspaces[pid] 指向了唯一标识符 pid 对应的地址空间

```
#define NumProcesses 256
#define NumSysProcesses 100
extern AddrSpace **addrspaces;
```

修改 AddrSpace 的构造函数和析构函数以实现 pid 的分配与销毁: 遍历整个数组, 如果当前为空则可以分配, 如果没有空间则分配失败



```

pid = -1;
for(i = (userprog ? NumSysProcesses : 0); i < (userprog ? NumProcesses : NumSysProcesses); i++) {
 if(addrspaces[i] == NULL) {
 addrspaces[i] = this;
 pid = i;
 break;
 }
}
if(pid == -1) {
 printf("ERROR!!!\nNo free process blocks!\n");
 ASSERT(FALSE);
}

```

当用户进程被销毁时释放该空间，即将 `addrspaces[pid]` 设置为 `NULL`

```

AddrSpace::~~AddrSpace()
{
 for(int i = 0; i < numPages; i++) {
 freePages->Clear(pageTable[i].physicalPage);
 }
 addrspaces[pid] = NULL;
 delete [] pageTable;
}

```

### 2.3. 实现 AdvancePC()

在 `OneInstruction (Instruction *instr)` 函数的 `case OP_SYSCALL` 中可以看出，当一条指令正常执行结束后，需要将 `PC` 推进，指向下一条指令，但其中，`RaiseException(SyscallException, 0);` 后不是一条 `break` 语句，而是一条 `return` 语句，原因是通常情况下，当处理完一个异常后需要重启这条指令。但系统调用异常是个特例，异常处理结束后指令不需要重启，因此需要系统调用处理程序中添加 `PC` 的推进操作，否则程序就会再次读入执行这条系统调用操作，造成无限循环。

在 `exception.cc` 中加入 `AdvancePC()` 手动实现 `PC=PC+1`，使得系统调用指令执行完后，`PC` 值会指向下一条指令所在地址。

```

void AdvancePC() {
 int pc = machine->ReadRegister(PCReg);
 int nextPC = machine->ReadRegister(NextPCReg);
 machine->WriteRegister(PrevPCReg, pc);
 machine->WriteRegister(PCReg, nextPC);
 machine->WriteRegister(NextPCReg, nextPC + 4);
}

```

### 2.4. 建立进程家族树

#### 2.4.1. 分析

当父进程执行系统调用 `Exit()` 退出时，会销毁已经结束的子进程，而未被结束的子进程会保持。此时需要维护一个进程家族树，来标识父子进程关系。当子进程结束时等待父进程，由父进程负责资源的回收；当子进程还没有结束而父进程已经结束则子进程会成为父进程的父进程的子进程。

#### 2.4.2. 具体实现：

在类 `Thread` 增加两个成员变量：`Thread` 类型的指针 `pThread` 和线性表 `cThreads`，前者用来表示该进程的父进程，后者用来表示该进程的子进程，绑定父子进程时则将子进程的父进程指针指向父进程，同时将子进程添加到父进程的子进程列表中，使用函数 `SetParentThread()` 和函数 `AppendChildThread()` 来实现。

```
private:
 Thread *pThread;
 List *cThreads;
public:
 void BindThread(Thread *childThread);
 void SetParentThread(Thread *parent) { pThread = parent; }
 Thread *GetParentThread() { return pThread; }
 void AppendChildThread(Thread *child) { cThreads->Append((void *)child); }
 List *GetChildThreads() { return cThreads; }
```

在 Nachos 系统初始化时定义一个根进程 MainThread，作为进程家族树中的树根，类似 UNIX 系统中的 1 号进程，所有进程的祖先均为该进程，该进程永远指向原 Nachos 系统初始化的 "main" 进程，这个线程永远不会进入就绪队列中，仅仅作为一个象征性的存在，不会映射地址空间也不会被系统调度执行

```
#ifdef USER_PROGRAM
 machine = new Machine(debugUserProg); // this must come first
 freePages = new BitMap(NumPhysPages);
 addrspaces = new AddrSpace*[NumProcesses];
 for(int i = 0; i < NumProcesses; i++) addrspaces[i] = NULL;
 MainThread = currentThread;
#endif
```

调度其他线程的执行时，在将 main 函数中原有的代码 currentThread->Finish() 改写为 MainThread->Yield() 调度用户进程运行，同时修改函数 Thread::Yield() 来避免将 MainThread 加入至就绪队列中

```
void
Thread::Yield ()
{
 Thread *nextThread;
 IntStatus oldLevel = interrupt->SetLevel(IntOff);

 ASSERT(this == currentThread);

 DEBUG('t', "Yielding thread \"%s\"\n", getName());

 nextThread = scheduler->FindNextToRun();
 if (nextThread != NULL) {
 if (this != MainThread) scheduler->ReadyToRun(this);
 scheduler->Run(nextThread);
 }
 (void) interrupt->SetLevel(oldLevel);
}
```

由 mian 线程运行用户程序时，需要重构一个专用的函数 StartProcess(char \*filename)，其中为用户程序开辟一个新的线程，为这个线程绑定地址空间，使用 Thread::Fork() 将函数 StartProcess (SpaceId pid) 加入到就绪队列，然后将该线程的父线程指向线程 MainThread，同时将该线程加入 MainThread 的子线程列表中。

```
#ifdef USER_PROGRAM
 if (!strcmp(*argv, "-x")) { // run a user program
 ASSERT(argc > 1);
 StartProcess(*(argv + 1));
 argCount = 2;
 }
```

```

StartProcess(char *filename)
{
 OpenFile *executable = fileSystem->Open(filename);
 AddrSpace *space;

 if (executable == NULL) {
 printf("Unable to open file %s\n", filename);
 return;
 }
 space = new AddrSpace(executable);

 Thread *thread = new Thread(filename);
 thread->BindSpace(space);
 thread->Fork(StartProcess, thread->GetSpaceId());
 MainThread->BindThread(thread);
 // space->Print();
 // currentThread->BindSpace(space);

 delete executable; // close file
}

```

StartProcess (SpaceId pid), 包括寄存器的初始化、页表的映射、程序运行

```

void StartProcess(SpaceId pid) {
 addrspace[pid]->InitRegisters();
 addrspace[pid]->RestoreState();
 machine->Run();
 ASSERT(FALSE);
}

```

当 Nachos 在 main 函数中执行语句 MainThread->Yield() 后该线程会被调度执行，在系统调用 Exec() 中也需要绑定父子关系，执行系统调用 Exec() 的进程是父进程，被调用的是子进程。

## 2.5. 实现 Exec()

### 2.5.1. Exec() 实现的思路

Exec() 的作用是以用户程序 name 为新的进程执行并且返回标识该进程地址空间的唯一标识符 Spaceid，运行流程如下：

1. 从内存中读取 name 并且打开该用户程序
2. 创建该用户程序对应的地址空间实例
3. 为用户程序分配新的线程，并且将刚刚创建的地址空间实例与该线程绑定，使用 Thread::Fork() 将函数 StartProcess (SpaceId pid) 加入到就绪队列
4. 运行 StartProcess (SpaceId pid)，包括寄存器的初始化、页表的映射、运行
5. 返回该进程地址空间的唯一标识符 pid
6. PC 增加 1

### 2.5.2. 具体实现。

重构的 StartProcess (SpaceId pid)，利用上文实现的 pid 访问线程的寄存器、页表等

其中绑定父子关系是为了方便构建进程树

```

void _Exec() {
 // 读取参数 该参数为在内存中的地址
 int addr = machine->ReadRegister(4);
 // 到内存中读取参数, 由于参数是字符串, 读到 '\0' 才停止
 char filename[LenOfFilename + 1];
 for(int i = 0; i <= LenOfFilename; i++) { ...
 DEBUG('s', "[SpaceId %d]: Syscall Exec(\"%s\")\n", currentThread->GetSpaceId(), filename);
 OpenFile *executable;
 if(!(executable = fileSystem->Open(filename))) { // 文件不存在 ...
 } else { // 文件存在
 AddrSpace *space = new AddrSpace(executable); // 分配地址空间
 delete executable;
 Thread *thread = new Thread(filename); // 创建线程
 thread->BindSpace(space); // 将用户进程映射到核心线程上
 thread->Fork(StartProcess, thread->GetSpaceId());
 currentThread->BindThread(thread); // 绑定父子关系
 machine->WriteRegister(2, thread->GetSpaceId()); // 返回 SpaceId 号
 }
 // PC+1->PC
 AdvancePC();
}

```

ExceptionHandler() 只负责判断系统调用类型并定位到相应的代码中

```

ExceptionHandler(ExceptionType which)
{
 int type = machine->ReadRegister(2);
 if(which != SyscallException) {
 printf("Unexpected user mode exception %d %d\n", which, type);
 ASSERT(FALSE);
 }

 switch(type) {
 case SC_Halt: _Halt(); break;
 case SC_Exit: _Exit(); break;
 case SC_Exec: _Exec(); break;
 case SC_Join: _Join(); break;
 case SC_Create: _Create(); break;
 case SC_Open: _Open(); break;
 case SC_Read: _Read(); break;
 case SC_Write: _Write(); break;
 case SC_Close: _Close(); break;
 case SC_Fork: _Fork(); break;
 case SC_Yield: _Yield(); break;
 case SC_Nachos_Fork: _N_Fork(); break;
 default:
 printf("Unexpected user mode exception %d %d\n", which, type);
 ASSERT(FALSE);
 }
}

```

## 2. 6. 实现 Exit()

### 2. 6. 1. 实现思路

系统调用 Exit() 用来结束一个用户程序, 同时给出该用户程序的退出状态。其具体思路如下: 1. 保存进程的退出状态 exitCode

2. 销毁已经结束的子进程
3. 将没有结束的子进程转交给父进程
4. 结束自己并调度其他的进程

### 2. 6. 2. 具体实现

定义相关的变量与函数: 在类 Thread 使用一个成员变量 int exitCode 来记录进程的退出状态以及相应的成员函数 int GetExitCode() { return exitCode; } 和 void SetExitCode(int status) { exitCode = status; }, 分别用于获取退出状态和设置退出状态

```

private:
 int exitCode;
public:
 int GetExitCode() { return exitCode; }
 void SetExitCode(int status) { exitCode = status; }

```

```
// Thread state
enum ThreadStatus { JUST_CREATED, RUNNING, READY, BLOCKED, TERMINATED };
```

在线程的状态列表中新增一个终止状态 `TERMINATED` 用于标志进程的结束，当进程结束自己的时候，只是将自己的状态设为 `TERMINATED`，等待父进程来销毁自己，然后在去调度其他线程即可

代码思路：首先从 4 号寄存器中读取退出码参数 `exitcode`，并将其设置为当前线程的退出状态，然后获取当前进程的子进程列表，然后循环遍历子进程，如果子进程仍未退出则进行回溯，将子进程转交给当前进程的父进程，如果子进程退出则将其销毁，最后退出调度其他进程

```
void _Exit() {
 int exitCode = machine->ReadRegister(4);

 DEBUG('s', "[SpaceId %d]: Syscall Exit(%d)\n", currentThread->GetSpaceId(), exitCode);

 currentThread->SetExitCode(exitCode);

 List *threadList = currentThread->GetChildThreads();
 ListElement *ptr = threadList->FirstElement();
 while(ptr != NULL) {
 Thread *thread = (Thread *)ptr->item;
 if(thread->getStatus() != TERMINATED) {
 currentThread->GetParentThread()->BindThread(thread);
 ptr = ptr->next;
 } else {
 ptr = threadList->Remove(ptr);
 delete thread;
 }
 }
 currentThread->Finish();
}
```

## 2.7. 实现 Join()

### 2.7.1. 分析

系统调用 `Join()` 的功能类似于 UNIX 系统中的函数 `wait()`：会暂时停止目前进程的执行，直到子进程结束。如果在调用 `Join()` 时子进程已经结束，则 `Join()` 会立即返回子进程结束状态值。

### 2.7.2. 具体实现

`Thread::Join (SpaceId pid)` 用于具体执行 `join` 的操作：首先循环检查子进程列表判断目标是否是子进程，如果不是返回 `FALSE`，如果是则获取子进程的状态并判断是否是 `TERMINATED`，如果不是说明子进程还没结束，阻塞父进程，如果子进程已经结束则获取子进程的退出码，最后销毁子进程，返回 `TRUE`

```

List *threadList = currentThread->GetChildThreads();
ListElement *ptr = threadList->FirstElement();
while(ptr != NULL) {
 Thread *thread = (Thread *)ptr->item;
 if(thread->GetSpaceId() == pid) {
 target = thread;
 break;
 }
 ptr = ptr->next;
}
if(!target) {
 interrupt->SetLevel(oldLevel);
 return FALSE;
}
waitProcessSpaceId = pid;
if(target->getStatus() != TERMINATED) Sleep();
waitProcessExitCode = target->GetExitCode();
threadList->Remove(ptr);
delete target;
interrupt->SetLevel(oldLevel);
return TRUE;

```

重构 Thread::Finish ()，用户进程结束自身时，首先检查一下父进程的 waitProcessSpaceId 属性，如果父进程的确在等待自己且处于阻塞状态，则唤醒之并加入就绪队列中，然后将自己的状态设为 TERMINATED，调度其他进程

```

Thread::Finish ()
{
 (void) interrupt->SetLevel(IntOff);
 ASSERT(this == currentThread);

 DEBUG('t', "Finishing thread \"%s\"\n", getName());
#ifdef USER_PROGRAM
 // 从等待队列中查找等待自己的进程
 if(pThread->waitProcessSpaceId == pid && pThread != MainThread) {
 scheduler->ReadyToRun(pThread);
 }
 setStatus(TERMINATED);
 // 调度其他线程
 Thread *nextThread;
 while((nextThread = scheduler->FindNextToRun()) == NULL) {
 interrupt->Idle();
 }
 scheduler->Run(nextThread);
#else
 threadToBeDestroyed = currentThread;
 Sleep(); // invokes SWITCH
#endif
}

```

实现系统调用 Join()：首先从 4 号寄存器中获取子进程号参数，然后调用 Thread::Join (SpaceId pid)并检查返回值，最后将子进程返回值写 2 号寄存器，PC 自增 1

```

void _Join() {
 // 读 pid
 SpaceId pid = machine->ReadRegister(4);
 DEBUG('s', "[SpaceId %d]: Syscall Join(%d)\n", currentThread->GetSpaceId(), pid);
 if(!currentThread->Join(pid)) {
 // pid 不是子进程 返回 0
 printf("Syscall Join(): %d is not child thread.\n", pid);
 machine->WriteRegister(2, 0);
 } else {
 // 将子进程的退出码返回
 machine->WriteRegister(2, currentThread->waitProcessExitCode);
 DEBUG('s', "[SpaceId %d]: Syscall Join(%d) return %d\n", currentThread->GetSpaceId(), pid, currentThread->waitProcessExitCode);
 }
 AdvancePC();
}

```



## 2.8. 实现 Yield()

### 2.8.1. 具体实现：直接使用类 Thread 中的函数 Yield()

```
void _Yield() {
 DEBUG('s', "[SpaceId %d]: Syscall Yield()\n", currentThread->GetSpaceId());
 currentThread->Yield();
 AdvancePC();
}
```

## 2.9. 实现 Fork()

### 2.9.1. Fork() 实现的思路

1. 为子进程开辟新的线程。
2. 根据父进程的地址空间，克隆一个新的地址空间并绑定在子进程中。
3. 父进程与子进程绑定父子关系。
4. 将相关的寄存器信息保存至子线程中
5. 当子线程被调度运行时恢复寄存器状态

### 2.9.2. 具体实现

为 AddrSpace 重构复制构造函数，使之可以根据父进程的地址空间来克隆一个新的地址空间，首先根据传入的地址空间对象的 pid 来为地址空间分配 SpaceId。然后检查是否有足够的空闲页面来存储新的地址空间。然后创建根据父进程的页面项构造子进程的页面项。对于每个页表项，找到一个空闲的物理页面，并将原来的物理页面的内容复制到新的物理页面，从而将父进程中的地址空间拷贝至子进程的地址空间中

```
AddrSpace::AddrSpace(AddrSpace *space) {
 int i, value, offset;
 int begin, end;
 pid = -1;
 begin = space->pid >= NumSysProcesses ? NumSysProcesses : 0;
 end = space->pid >= NumSysProcesses ? NumProcesses : NumSysProcesses;
 for(i = begin; i < end; ++i) { //分配SpaceId ...
 if(numPages > freePages->NumClear()) { //是否有足够的空闲页面 ...
 numPages = space->numPages;
 ASSERT(numPages <= freePages->NumClear());
 DEBUG('a', "Initializing forked address space\n");
 pageTable = new TranslationEntry[numPages];
 for (i = 0; i < numPages; i++) {
 pageTable[i] = space->pageTable[i];
 pageTable[i].physicalPage = freePages->Find();
 int physicalAddr1 = space->pageTable[i].physicalPage * PageSize;
 int physicalAddr2 = pageTable[i].physicalPage * PageSize;
 memcpy(&machine->mainMemory[physicalAddr2], &machine->mainMemory[physicalAddr1], PageSize);
 }
 ref = 0;
 Print();
 }
 }
}
```

### 实现系统调用 Fork()

```
void _Fork() {
 DEBUG('s', "[SpaceId %d]: Syscall Fork()\n", currentThread->GetSpaceId());
 Thread *thread = new Thread("forked");// 新建一个线程并绑定地址空间
 thread->BindSpace(new AddrSpace(currentThread->space));
 currentThread->BindThread(thread); // 绑定父子关系
 currentThread->SaveUserState();// 保存当前寄存器状态
 machine->WriteRegister(2, 0);// 对子进程返回 0
 AdvancePC();
 thread->SaveUserState();// 将当前寄存器状态保存至子进程中
 thread->Fork(StartForkedProcess, (_int)thread);
 currentThread->RestoreUserState();// 恢复寄存器状态
 machine->WriteRegister(2, thread->GetSpaceId());// 对父进程返回子进程 SpaceId
 AdvancePC();
}
```

## 2. 10. 基于 FILESYS\_STUB 所实现的文件系统

### 2. 10. 1. Creat()

读出寄存器中传入的文件名, 调用文件系统的 OpenForWrite(filename) 函数创建一个文件即可

```
void _Create() {
 // 读地址
 int addr = machine->ReadRegister(4);
 // 读文件名
 char *filename = new char[LenOfFilename + 1];
 for(int i = 0; i <= LenOfFilename; ++i) {
 machine->ReadMem(addr + i, 1, (int *)&filename[i]);
 if(filename[i] == '\0') break;
 }
#ifdef FILESYS_STUB
 int fd = OpenForWrite(filename);
 DEBUG('s', "[SpaceId %d]: Syscall Create(\"%s\") fd=%d\n", currentThread->GetSpaceId(), filename, fd);
 Close(fd);
#else //FILESYS
#endif
 delete filename;
 AdvancePC();
}
```

### 2. 10. 2. Open()

打开一个文件并返回文件的 OpenFileId。因此我们读出寄存器中传入的文件名, 调用文件系统的 OpenForWrite(filename) 打开一个文件 即可。

```
void _Open() {
 // 读地址
 int addr = machine->ReadRegister(4);
 // 读文件名
 char *filename = new char[LenOfFilename + 1];
 for(int i = 0; i <= LenOfFilename; ++i) {
 machine->ReadMem(addr + i, 1, (int *)&filename[i]);
 if(filename[i] == 0) break;
 }
#ifdef FILESYS_STUB
 int fd = OpenForReadWrite(filename, FALSE);
 DEBUG('s', "[SpaceId %d]: Syscall Open(\"%s\") fd=%d\n", currentThread->GetSpaceId(), filename, fd);
 machine->WriteRegister(2, fd);
#else //FILESYS
#endif
 delete filename;
 AdvancePC();
}
```

### 2. 10. 3. Write()

将 buffer 中的前 size 个字节写入 id 所对应的文件中。因此我们依次从寄存器中读出传入的各参数, 调用 OpenFile 类的 WriteAt 函数将数据写入一个文件即可。



```

void _Write() {
 // 读取地址、大小和文件描述符
 int addr = machine->ReadRegister(4);
 int size = machine->ReadRegister(5);
 int fd = machine->ReadRegister(6);
 DEBUG('s', "[SpaceId %d]: Syscall Write() to file %d from addr 0x%x\n", currentThread->GetSpaceId(), fd, addr);
 char *buffer = new char[size + 1];
#ifdef FILESYS_STUB
 // 打开文件
 OpenFile *openfile = new OpenFile(fd);
 ASSERT(openfile != NULL);
 // 从内存中读取要写入的数据至 buffer 中
 for(int i = 0; i < size; ++i) {
 machine->ReadMem(addr + i, 1, (int *)&buffer[i]);
 }
 buffer[size] = 0;
 DEBUG('s', "\twrite %d bytes, content: %s\n", size, buffer);
 // 将 buffer 中的内容写入文件
 openfile->WriteAt(buffer, size, fd == 1 ? 0 : openfile->Length());
}

```

#### 2. 10. 4. Read()

从 OpenFileId 所对应的文件中 读取 size 个字节到 buffer 中，返回值为具体读取的有效字节数。因此我们依次从寄存器中 读出传入的各参数，调用 OpenFile 类的 Read 函数以及 Machine 类的 WriteMem 函数读出数据并写入 buffer 来完成此系统调用。

```

void _Read() {
 // 读地址、大小和文件描述符
 int addr = machine->ReadRegister(4);
 int size = machine->ReadRegister(5);
 int fd = machine->ReadRegister(6);
 DEBUG('s', "[SpaceId %d]: Syscall Read() from file %d to addr 0x%x\n", currentThread->GetSpaceId(), fd, addr);
 char *buffer = new char[size + 1];
#ifdef FILESYS_STUB
 // 打开文件
 OpenFile *openfile = new OpenFile(fd);
 ASSERT(openfile != NULL);
 // 从文件中读内容至 buffer 中,大小不会超过文件的长度
 int num = openfile->Read(buffer, min(size, openfile->Length()));
 buffer[num] = 0;
 // 将 buffer 中的内容写至内存中
 for(int i = 0; i <= num; ++i) {
 machine->WriteMem(addr + i, 1, buffer[i]);
 }
 DEBUG('s', "\tread %d bytes, content: %s\n", num, buffer);
 // 放回读取的字节数
 machine->WriteRegister(2, num);
}

```

#### 2. 10. 5. Close()

文件读写已结束，关闭文件。因此我们从寄存器中读出传入的 OpenFileId，调用 Close() 关闭该文件即可。

```

void _Close() {
 // 读文件描述符
 int fd = machine->ReadRegister(4);
 DEBUG('s', "[SpaceId %d]: Syscall Close(%d)\n", currentThread->GetSpaceId(), fd);
#ifdef FILESYS_STUB
 // 关闭文件
 Close(fd);
#else// FILESYS
#endif
 AdvancePC();
}

```

### 3. 系统调用的测试

#### 3.1. 综合测试

涉及到的系统调用有 Exec()、Join()、Yield() 以及文件系统调用 Create()、Open()、Write() 与 Close()

```

//exec.c
#include "syscall.h"

int main() {
 int fd, pid;
 pid = Exec("../test/exit.noff");
 Join(pid);
 fd = Open("AAA");
 Write("I am exec\n", 10, fd);
 Close(fd);
 Exit(0);
}

```

```

//exit.c
#include "syscall.h"

int main() {
 int fd;
 Create("AAA");
 fd = Open("AAA");
 Yield();
 Write("I am exit\n", 10, fd);
 Close(fd);
 Exit(0);
}

```

运行 ./nachos -d s -x ../test/exec.noff

```

[SpaceId 100]: Syscall Exec("../test/exit.noff")
[SpaceId 100]: Syscall Join(101)
[SpaceId 101]: Syscall Create("AAA") fd=3
[SpaceId 101]: Syscall Open("AAA") fd=3
[SpaceId 101]: Syscall Yield()
[SpaceId 101]: Syscall Write() to file 3 from addr 0x174
 write 10 bytes, content: I am exit

[SpaceId 101]: Syscall Close(3)
[SpaceId 101]: Syscall Exit(0)
[SpaceId 100]: Syscall Join(101) return 0
[SpaceId 100]: Syscall Open("AAA") fd=3
[SpaceId 100]: Syscall Write() to file 3 from addr 0x188
 write 10 bytes, content: I am exec

[SpaceId 100]: Syscall Close(3)
[SpaceId 100]: Syscall Exit(0)
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

```

结果分析：执行 `exec.c`，`exec.c` 调用了系统调用 `Exec()` 来装在用户程序 `exit.c` 然后 `exec.c` 会因执行 `Join()` 而挂起等待用户程序 `exit.c` 的退出。用户程序 `exec.c` 首先会创建一个文件 `AAA` 并且打开它，在此之后调用了 `Yield()` 而将自己挂起，由于是 FCFS 调度模式，目前没有可以调度的进程，故用户程序 `exit.c` 继续执行。用户程序在往文件 `AAA` 写入 “I am exit” 共 10 个字节的数据之后关闭了文件，以 0 为退出码退出。接下来用户程序 `exec.c` 被唤醒，会打开文件 `hrz` 并往里写入 “I am exec” 共 10 个字节的数据，关闭文件，退出。

### 3.2. 测试 (fork)

```

#include "syscall.h"
int fd;
void test1() {
 int i;
 char s[3] = "A ";
 for(i = 0; i < 3; ++i) {
 s[0] = 'A' + i;
 Write(s, 2, fd);
 }
}
void test2() {
 int j;
 char s[3] = "a ";
 for(j = 0; j < 5; ++j) {
 s[0] = 'a' + j;
 Write(s, 2, fd);
 }
}
int main() {
 Create("AAA");
 fd = Open("AAA");
 Nachos_Fork(test1);
 Nachos_Fork(test2);
 return 0;
}

```

这个程序会创建两个线程，线程 1 会连续向文件 `AAA` 写入大写字母 `ABC`，线程 2 会

连续向文件写入小写字母 abcde

运行 ./nachos -d s -x ../test/fork.noff

```
wnb@wnb-virtual-machine:~/OS4.0/lab7-8$./nachos -d s -x ../test/fork.noff
[SpaceId 100]: Syscall Create("AAA") fd=3
[SpaceId 100]: Syscall Open("AAA") fd=3
[SpaceId 100]: Syscall Nachos_Fork(240)
[SpaceId 100]: Syscall Nachos_Fork(408)
[SpaceId 100]: Syscall Exit(0)
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: A
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: B
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: C
[SpaceId 100]: Syscall Exit(0)
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: a
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: b
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: c
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: d
[SpaceId 100]: Syscall Write() to file 3 from addr 0x6e0
write 2 bytes, content: e
[SpaceId 100]: Syscall Exit(0)
```

```
wnb@wnb-virtual-machine:~/OS4.0/lab7-8$ cat AAA
```

```
A B C a b c d e wnb@wnb-virtual-machine:~/OS4.0/lab7-8$
```

查看结果可知结果正确,最终输出文件 AAA 中出现了大写字母 ABC 和了小写字母 abcde

## 结论分析与体会:

实验八要求我们在实验七的基础上实现 Exec/Exit 两个系统调用,完成 Join 系统调用的构建进程树方法和 Fork 的系统调用的实现方法,最后比较完善的实现了 Nachos 系统调用。通过实验八,我深入理解 Nachos 系统调用机制和应用程序从编写到执行的完整过程,掌握了从分析系统调用的功能、到思考 实现功能的方法、再到具体的编写代码的过程,使我的能力有很大的提升。

在实现 Nachos 操作系统的线程同步、文件系统扩展、地址空间扩展和系统调用实现的过程中,我回顾了上学期所学的进程管理、内存管理、存储管理等相关知识,一步步地加深了对操作系统的理解。